

```

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from keras.datasets import imdb

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)

data = np.concatenate((X_train, X_test), axis=0)
label = np.concatenate((y_train, y_test), axis=0)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)

print("Review is ", X_train[0])
print("Review is ", y_train[0])

vocab = imdb.get_word_index()
print(vocab)

print("Categories:", np.unique(label))
print("Number of unique words:", len(np.unique(np.hstack(data))))

length = [len(i) for i in data]
print("Average Review length:", np.mean(length))
print("Standard Deviation:", round(np.std(length)))

print("Label:", label[0])
print("Label:", label[1])
print(data[0])

index = imdb.get_word_index()
reverse_index = dict([(value, key) for (key, value) in index.items()])

decoded = " ".join([reverse_index.get(i - 3, "#") for i in data[0]])
print(decoded)

def vectorize(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension), dtype=np.float16)
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1
    return results

```

```

data = vectorize(data)
label = np.array(label).astype("float16")

sns.set_style("darkgrid")
labelDF = pd.DataFrame({
    'label': label.astype("float32")
})
sns.countplot(x='label', data=labelDF)
plt.show()

X_train, X_test, y_train, y_test = train_test_split(
    data,
    label,
    test_size=0.20,
    random_state=1
)

print(X_train.shape)
print(X_test.shape)

from keras import models
from keras import layers
model = models.Sequential()

model.add(layers.Dense(50, activation="relu", input_shape=(10000,)))
model.add(layers.Dropout(0.3))
model.add(layers.Dense(50, activation="relu"))
model.add(layers.Dropout(0.2))
model.add(layers.Dense(50, activation="relu"))
model.add(layers.Dense(1, activation="sigmoid"))
model.summary()

import tensorflow as tf
callback = tf.keras.callbacks.EarlyStopping(
    monitor='loss',
    patience=3
)

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

```
results = model.fit(
    X_train,
    y_train,
    epochs=2,
    batch_size=500,
    validation_data=(X_test, y_test),
    callbacks=[callback]
)
```

```
print(np.mean(results.history["val_accuracy"]))
```

```
score = model.evaluate(
    X_test,
    y_test,
    batch_size=500
)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
```

```
print(results.history.keys())
```

```
plt.plot(results.history['accuracy'])
plt.plot(results.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```

```
plt.plot(results.history['loss'])
plt.plot(results.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```